

Inertial Racing: 惯性赛车环境中的强化学习算法对比研究

数学科学学院 吴卓轩 2501210105

2026 年 6 月 9 日

摘要

强化学习中有模型方法与无模型方法在样本效率上存在显著差异，但在具体任务中的量化对比研究尚不充分。本文实现了一个具有惯性动力学的网格世界赛车环境（Inertial Racing），并通过广度优先搜索（BFS）将状态空间最高压缩至理论值的不到10%。我们系统比较了价值迭代（有模型）与Q-learning（无模型）在9个不同规模赛道上的性能，提出了一种探索时过滤已知死亡动作的改进策略，并通过公平早停实验（检测不计时）和消融实验进行了验证。实验结果表明，价值迭代收敛极快（最大赛道256轮，4.7分钟），而Q-learning在给定足够幕数下也能收敛到相同的最优代价，但需要3个数量级以上的环境交互。死亡动作过滤使Q-learning的收敛幕数减少约15%，但训练时间并未显著下降，原因在于不过滤模式中大量短命幕降低了平均步长，且过滤模式每步引入额外计算开销。状态空间压缩使得大规模赛道（理论状态数3390万）的价值迭代成为可能（压缩率99.36%）。本文还详细分析了两种算法的学习动态：价值迭代从终点向起点传播价值，而Q-learning从起点向终点逐步探索。

本研究的代码和赛道数据已开源：https://github.com/daidaitx/Inertial_Racing。

关键词：强化学习；惯性赛车；价值迭代；Q-learning；状态空间压缩

1 引言

强化学习（Reinforcement Learning, RL）在游戏、机器人、自动驾驶等领域取得了显著成功。其中，有模型方法（Model-based RL）利用环境模型进行规划，通常具有极高的样本效率；而无模型方法（Model-free RL）通过与环境的直接交互学习策略，普适性强但样本效率较低。在具体任务中量化对比这两类方法的性能差异，对于理解它们的适用场景具有重要意义。

惯性赛车（Inertial Racing）是一个经典的具有动量（Momentum）的网格世界问题：赛车在离散网格上行驶，每一步的移动不仅取决于当前选择的动作，还受到上一时刻速度的影响。这类问题在强化学习教学中常被用作展示状态空间设计、值函数传播等概念的案例。然而，现有的实现往往忽略了大状态空间的处理，且缺乏对算法学习动态的深入分析。

本文的主要贡献如下：

1. 实现了一个完整的惯性赛车环境，通过广度优先搜索（BFS）从初始状态出发构建可达状态空间，将理论状态数压缩至原来的不到 10%，使得大规模赛道（如图 1 所示的 100×140 赛道，可行驶格子数 5823）的价值迭代成为可能。
2. 系统比较了价值迭代（有模型）与 Q-learning（无模型）在 9 个不同规模赛道上的性能，包括收敛速度、最终策略质量、训练时间等指标。
3. 提出了一种探索时过滤已知死亡动作的 Q-learning 改进策略，并通过公平早停实验（检测不

计入训练时间)验证了其对本样本效率的提升(幕数减少约 15%),同时分析了其计算开销的影响。

4. 详细分析了两种算法的学习动态:价值迭代的价值函数从终点向起点传播,而 Q-learning 的价值函数从起点向终点逐步蔓延,揭示了它们截然不同的学习模式。

本文的组织结构如下:第 2 节形式化描述问题;第 3 节介绍实验环境和设置;第 4 节阐述算法原理与实现细节;第 5 节展示实验结果与分析;第 6 节讨论相关工作和局限性;第 7 节总结全文。

custom赛道布局



图 1: custom 赛道布局 (示例), 不同颜色代表普通道路、起点、终点和障碍物; 终点格子上标注的数字为深入奖励值。

2 问题建模

2.1 马尔可夫决策过程

我们将惯性赛车问题建模为离散时间、有限状态的马尔可夫决策过程 (Markov Decision Process, MDP):

- **状态空间 S :** 每个状态由当前坐标 cur 和上一时间步的坐标 $prev$ 组成, 即 $s = (cur, prev)$ 。初始时 $cur = prev$ (速度为零)。
- **动作空间 A :** 9 个动作, 对应 8 个邻域方向和原地不动, 即

$$A = \{(dx, dy) \mid dx, dy \in \{-1, 0, 1\}\}.$$

- **状态转移:** 给定当前状态 $s = (cur, prev)$ 和动作 $a = (dx, dy)$, 首先根据速度 $v = cur - prev$ 计算惯性延伸点 $temp = cur + v$, 然后下一位置 $next_cur = temp + a$ 。转移的详细几何关系见图 2。

- **奖励函数：**每一步的即时奖励为 -10 （步惩罚）；若 next_cur 恰好位于终点区域，则额外获得深入奖励 $R_{\text{goal}} \in [0, 10]$ （数值越大表示终点格子越深入内部）；若 next_cur 为障碍物或超出边界，则奖励为 $-\infty$ （死亡），该幕立即终止。
- **折扣因子** $\gamma = 1.0$ （情节式任务，无折扣）。

2.2 惯性动力学与碰撞检测

惯性动力学的核心是速度的延续。图 2 的左子图展示了从 prev 到 cur 的速度矢量（蓝色实线箭头），以及由此推出的惯性延伸点 temp （蓝色虚线箭头）。右子图显示了从 temp 出发的 9 个候选动作（灰色点线箭头），每个动作对应的最终位置 next_cur 用不同颜色的边框标出：淡绿色区域表示可达（奖励 -10 ），淡红色区域表示不可达（奖励 $-\infty$ ）。路径的可行性通过 Bresenham 直线算法检查从 cur 到 next_cur 的直线是否穿过障碍物。

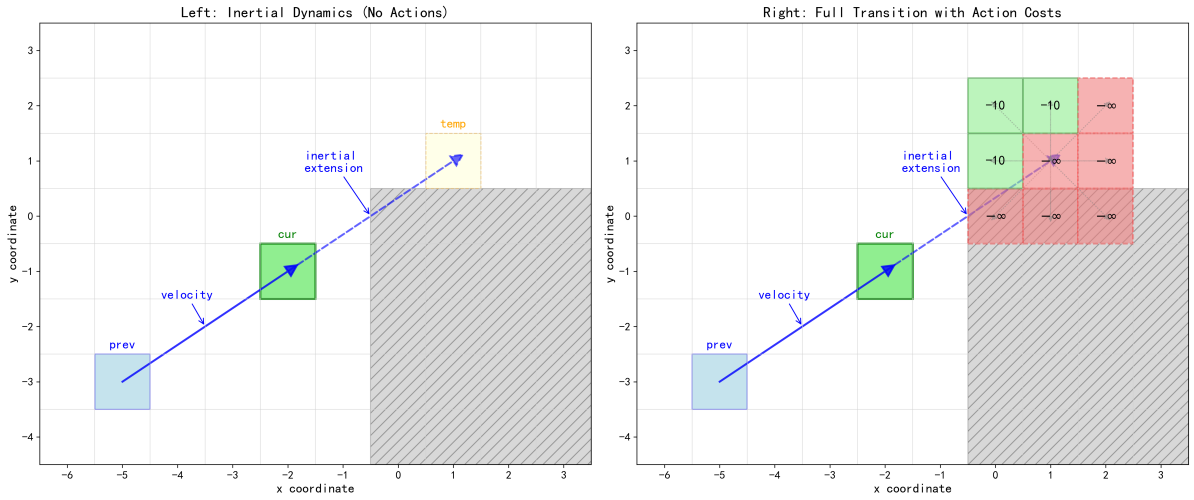


图 2: 惯性动力学示意图。左：速度与惯性延伸；右：从 temp 出发的 9 个动作及其可达性。

2.3 状态空间压缩

若直接使用所有可行行驶格子的笛卡尔积作为状态空间，则理论状态数为 N_{drivable}^2 。对于大规模赛道（如 *custom*，可行行驶格子数 5823），理论状态数高达 3.39×10^7 ，难以进行价值迭代。然而，由于惯性约束和连通性，大部分状态是不可达的。本文采用广度优先搜索（BFS）从所有初始状态 (c, c) 出发，仅保留可达的 $(\text{cur}, \text{prev})$ 对。实验表明，在本文测试的 9 个赛道中，BFS 将状态空间压缩至理论值的 $0.64\% \sim 32.10\%$ ，压缩率介于 $67.90\% \sim 99.36\%$ 之间（具体数据见表 1）。这使得价值迭代在 *custom* 赛道上仅需 256 轮即可收敛。

2.4 终点深入奖励

为了不仅靠运动步数评判轨迹优劣，我们应鼓励智能体冲线前尽可能加速。我们通过深入奖励鼓励智能体选择更深处的终点格子做到了这一点。我们根据终点格子到可行行驶区域边缘的 BFS 距离分配奖励。具体地，首先找到所有与可行行驶非终点格子相邻的终点格子，赋予奖励 0；然后向内 BFS，每深入一层奖励增加 $10/D_{\text{max}}$ ，其中 D_{max} 为最大深度。最终奖励值标注在赛道布局图的终点格子上（例如图 5 中终点区域上的数字）。这种设计使得智能体在靠近终点区域时有更细粒度的偏好，从而得到更优的路径。

此外，对于一个成功到达终点的轨迹，每步奖励为 -10 ，冲线奖励视冲线深度给予（不超过单步奖励的相反数 10），那么总奖励将恒非正，使得总奖励的绝对值可以合理地近似看为赛车的总赛道用时（或总代价）。

3 实验环境与设置

3.1 赛道与环境统计

本文共使用 9 个不同规模的赛道进行实验，其基本参数如表 1 所示。所有赛道的网格均采用四类单元格：普通道路（白色，记为 0）、起点（绿色，记为 1）、终点（红色，记为 2）和障碍物（黑色，记为 3）。可行驶格子数为普通道路、起点和终点格子之和。理论状态数为可行驶格子数的平方（即所有可能的 $(cur, prev)$ 对），而 BFS 合法状态数是从所有初始状态出发实际可达的状态数。压缩率定义为 $1 - (\text{BFS 状态数})/(\text{理论状态数})$ 。从表中可见，BFS 能够将状态空间压缩至理论值的 0.64%~32.10%，压缩率高达 67.90%~99.36%，这使大规模赛道的价值迭代成为可能。

表 1: 9 个赛道的基本统计信息

赛道名称	尺寸 ($H \times W$)	可行驶格子数	理论状态数	BFS 状态数	压缩率
toy	3×4	9	81	26	67.90%
straight	1×87	87	7,569	1,199	84.16%
short_straight	1×12	12	144	36	75.00%
U_shape	15×18	198	39,204	4,044	89.68%
U_narrow_shortcut	15×18	204	41,616	4,321	89.62%
S_shape	16×22	224	50,176	4,652	90.73%
variant_U_shape	13×31	391	152,881	20,965	86.29%
random	44×44	1,493	2,229,049	47,447	97.87%
custom	100×140	5,823	33,907,329	216,560	99.36%

图 3 展示了各赛道的状态空间压缩率。

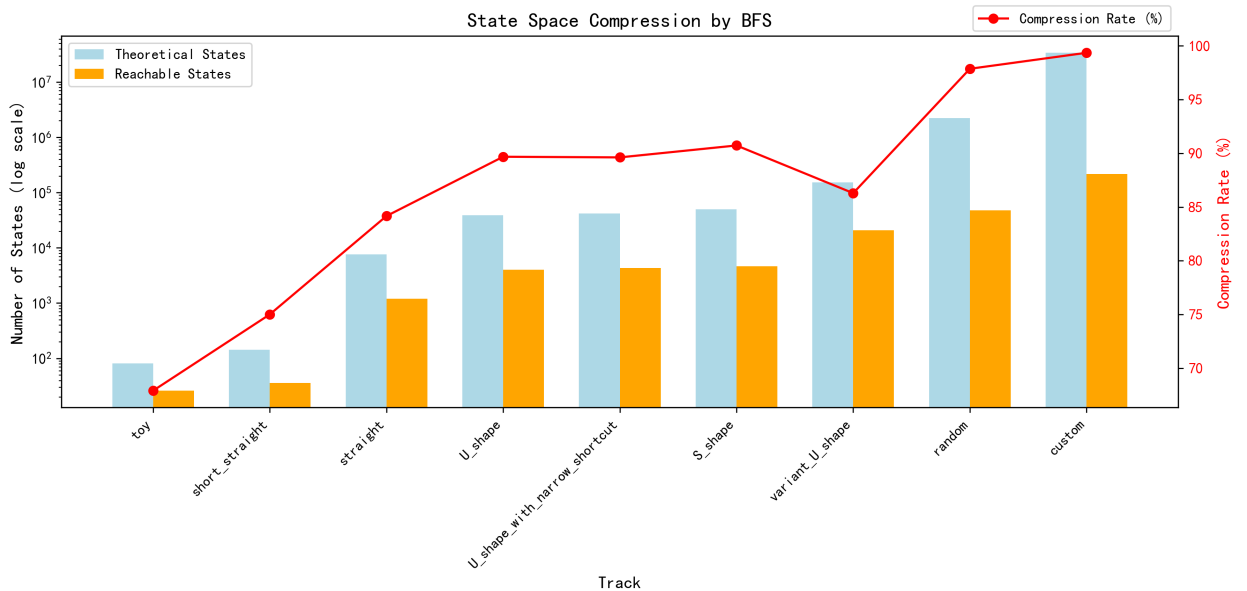


图 3: 9 个赛道的状态空间压缩率

图 5 展示了两个典型赛道的布局：U_shape（左）和 S_shape（右）。图中红色终点格子上标注的数字即为深入奖励值（见第 2.4 节），数值越高表示该终点格子离可行驶区域边缘越远，奖励越大。后续实验将多次引用这些赛道。

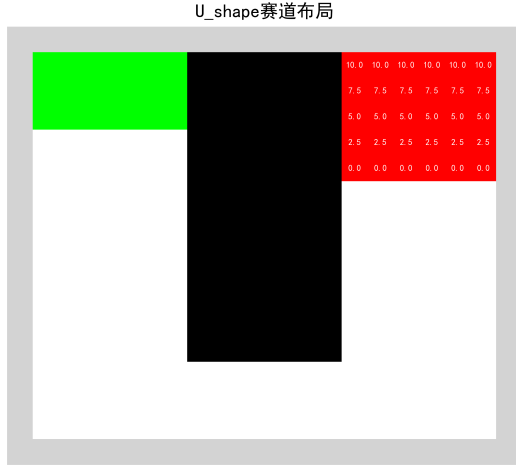


图 4: U_shape 赛道布局

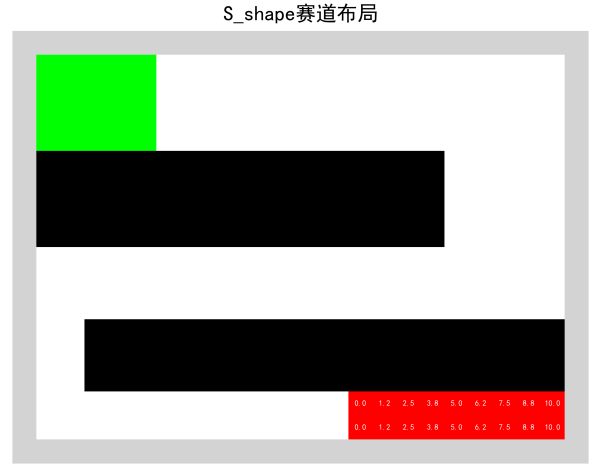


图 5: S_shape 赛道布局

3.2 算法超参数与运行设置

价值迭代: 采用异步 Gauss-Seidel 就地更新, 收敛阈值 $\theta = 10^{-6}$, 最大迭代次数 10000, 折扣因子 $\gamma = 1.0$ 。收敛时记录迭代轮数和耗时。

Q-learning 主实验: 学习率 $\alpha = 1.0$, 初始探索率 $\varepsilon = 1.0$, 指数衰减至 $\varepsilon_{\min} = 0.1$, 衰减系数 $\beta = \varepsilon_{\min}^{1/(0.9 \cdot N)}$, 其中 N 为预设总幕数。每个赛道的预设幕数通过预实验确定为足够大的值 (见表 2), 确保算法能收敛到最优。重复次数: 小赛道 (toy, short_straight) 10 次, 中赛道 (straight, U_shape, U_shape_narrow_shortcut, S_shape) 5 次, 大赛道 (variant_U_shape, random) 3 次, custom 赛道仅运行 1 次 (规模过大, 限于时间成本)。

Q-learning 公平早停实验: 为了获得 Q-learning 真实收敛所需的幕数和纯训练时间, 我们单独运行了早停实验 (除 custom 外所有赛道各 10 次)。每完成 1 个 episode 后, 执行一次贪婪策略评估 (检测所有起点), 若当前最优代价 (即 $-\max$ 总回报) 等于价值迭代给出的已知最优值, 则立即停止。检测过程不计入训练时间, 从而确保记录的时间仅包含网络交互与 Q 表更新。该实验的结果用于对比样本效率和时间效率。

Q-learning 消融实验: 固定赛道 U_shape_with_narrow_shortcut, 对比两种探索模式:

- **filter:** ε -greedy 探索时, 只从已知非死亡动作 ($Q > -\infty$) 中随机选择。
- **no_filter:** 探索时允许选择所有 9 个动作 (包括可能导致死亡的动作)。

两组除探索策略外其余参数完全相同, 各重复 50 次, 早停条件为最优代价 ≤ 100 (该值由价值迭代预先算出)。

3.3 评价指标

我们主要关注以下指标:

- **最优代价:** 即 $-\max$ 总回报, 数值越小代表轨迹越优 (步数少且冲线奖励高)。
- **收敛所需步数:** 价值迭代为迭代轮数; Q-learning 为主实验的固定幕数 (仅用于验证最终性能) 或早停实验的收敛幕数。
- **训练时间:** 价值迭代为总耗时; Q-learning 为早停实验中纯训练时间 (不含检测)。
- **成功率:** 算法能否找到一条从起点到终点的有效路径。

4 方法

4.1 价值迭代（有模型）

价值迭代利用环境已知的转移模型直接计算最优价值函数。本文采用异步 Gauss-Seidel 就地更新方式，算法流程如 Algorithm 1 所示。由于所有转移已在环境构建时预计算并存储在表格 $\mathcal{T}$ 中 ($\mathcal{T}[s][a] = (\text{type}, \text{data})$)，每次迭代只需遍历所有状态并更新 $V(s)$ 即可。

算法 1: 异步价值迭代 (Gauss-Seidel)

输入 : 环境转移表 $\mathcal{T}$, 折扣因子 $\gamma = 1.0$, 收敛阈值 $\theta = 10^{-6}$
输出 : 最优价值函数 V , 最优策略 π

- 1 初始化 $V(s) = 0$ 对所有 $s \in \mathcal{S}$;
- 2 **repeat**
- 3 **for** $s \in \mathcal{S}$ **do**
- 4 $v_{\text{old}} \leftarrow V(s)$;
- 5 $V(s) \leftarrow \max_{a \in \mathcal{A}} (-10 + \gamma \cdot V(s'))$, 其中 s' 由 $\mathcal{T}(s, a)$ 给出 (若为终点则 $V(s') = 0$, 若为死亡则值为 $-\infty$);
- 6 $\Delta \leftarrow |V(s) - v_{\text{old}}|$;
- 7 **end**
- 8 **until** $\max_s |\Delta V(s)| < \theta$;
- 9 根据最终 V 提取贪婪策略 $\pi(s) = \arg \max_a (-10 + \gamma \cdot V(s'))$;

由于状态空间经过 BFS 压缩后规模可控 (最大约 $2.1\text{e}5$ 个状态), 价值迭代通常只需数十至数百轮即可收敛 (见表 3)。

4.2 Q-learning (无模型)

Q-learning 不依赖环境模型, 直接通过与环境交互更新 Q 值。本文采用标准的 off-policy 更新规则:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right),$$

其中 $\alpha = 1.0$ 为学习率, $\gamma = 1.0$ 为折扣因子。算法使用 ε -greedy 策略平衡探索与利用: 以概率 ε 随机选择动作, 否则选择当前 Q 值最大的动作。 ε 从 1.0 指数衰减至 0.1, 衰减系数 $\beta = 0.1^{1/(0.9N)}$, N 为预设总幕数。

学习率选择为 $\alpha = 1.0$, 是因为我们的环境是确定性的, 不需要通过小步长磨去随机误差。这样可以在很大程度上加速算法的收敛。

折扣因子选择为 $\gamma = 1.0$, 是因为我们已经通过每步奖励-10和终点深入奖励惩罚了运动步数, 不再需要折扣因子的作用。

4.2.1 死亡动作过滤

在确定性环境中, 某些动作必然导致撞墙死亡 (奖励 $-\infty$)。一旦某个状态-动作对 (s, a) 的 Q 值变为 $-\infty$, 该动作再被选择只会重复死亡, 毫无意义。因此, 我们在 ε -greedy 的探索阶段加入过滤机制: 当需要随机选择动作时, 只从那些当前 Q 值 $> -\infty$ 的动作中均匀采样 (若所有动作均死亡, 则退化到全部动作)。这避免了大量无效的死亡幕, 将探索资源集中于有希望的动作上。

死亡奖励保持 $-\infty$ 不变, 这是合理的设计: 因为该动作在确定性环境下永远不可行, 没有再次尝试的必要。Q-learning 的 off-policy 性质使得 $-\infty$ 不会污染其他动作的价值 (max 操作会自然忽略它们)。

4.2.2 公平早停实验

为了获得 Q-learning 收敛所需的真实幕数和纯训练时间，我们还设计了早停实验（结果见第 5.4 节）。每完成一个 episode，算法立即对所有起点进行一次贪婪策略评估（模拟至多 1000 步），若当前最优代价达到价值迭代预先算出的理论最优值，则停止训练。评估过程不计入训练时间，仅记录交互与 Q 表更新的耗时。这样得到的收敛幕数不受检测频率影响，时间也排除了评估开销。

4.3 实现细节与代码结构

所有算法均用 Python 实现，核心类包括：

- **InertialRacingEnv**: 环境主类，负责状态空间 BFS 构建、转移表预计算、提供 `step_raw` 接口。
- **QLearningAgent**: Q-learning 智能体，支持死亡动作过滤与早停评估。
- **value_iteration**: 价值迭代函数，可直接利用环境的转移表。

可视化工具基于 Matplotlib 和 Seaborn，可生成赛道布局、价值热力图、学习曲线、轨迹图等。所有实验数据和日志自动保存至 CSV 文件，便于后续分析。

5 实验结果

5.1 可视化示例：S_shape 与 custom 赛道的学习动态

为了直观展示两种算法的学习过程，我们选取 S_shape（典型弯曲赛道）和 custom（最大规模赛道）作为代表。图 6 和 7 分别给出了价值迭代和 Q-learning 最终的价值热力图。可以看出，两种算法获得的价值分布非常接近，且最优轨迹（图 8、9）完全一致，验证了 Q-learning 在给定足够幕数下也能收敛到全局最优。

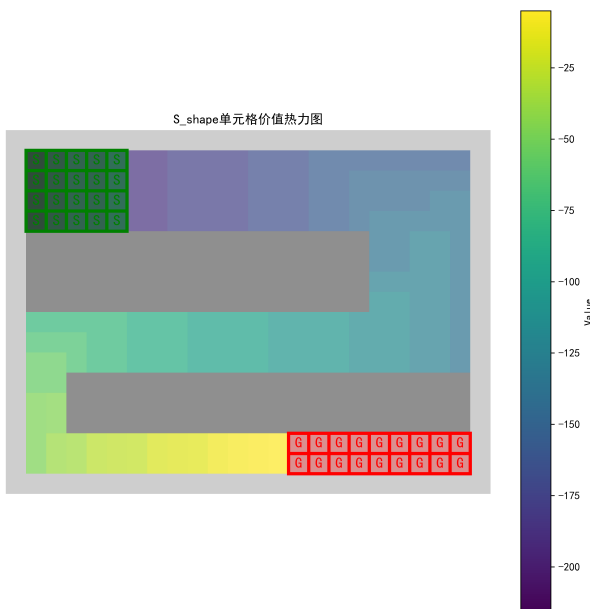


图 6: S_shape 价值迭代热力图

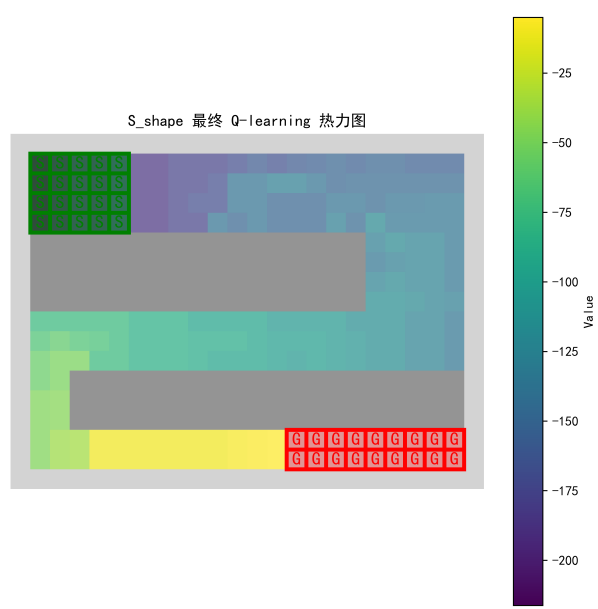


图 7: S_shape Q-learning 热力图

更深入的学习动态可以从 custom 赛道的演化图观察。图 10 展示了价值迭代过程中价值函数的传播：初始全零，随后从终点区域开始，价值逐渐向起点蔓延，符合价值迭代的“倒推”特性。对应的策略演化（图 11）也显示，早期策略杂乱无章，直到某一轮迭代后突变出完整的最优路径——这是因为价值一旦传播到起点，贪婪策略就能直接得到全局最优。

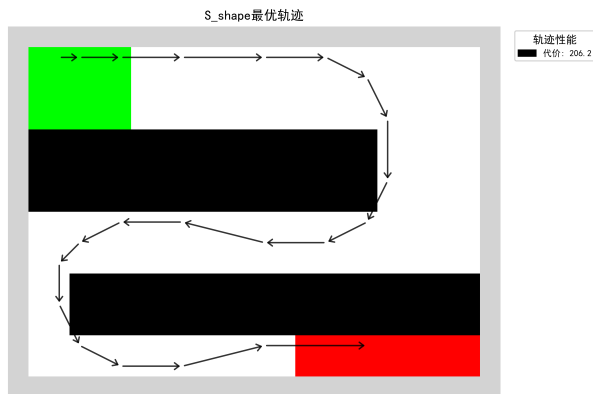


图 8: S_shape 价值迭代最优轨迹

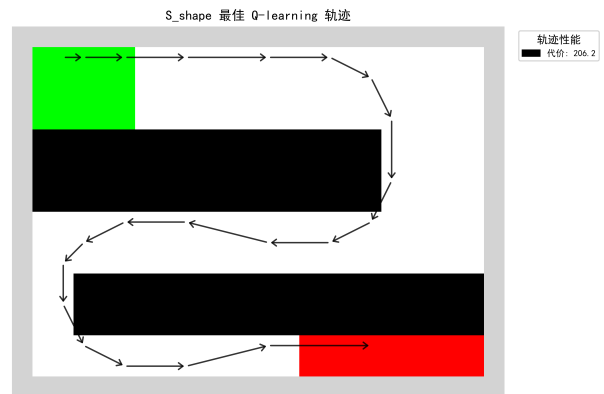


图 9: S_shape Q-learning 最优轨迹

custom单元格价值热力演化图

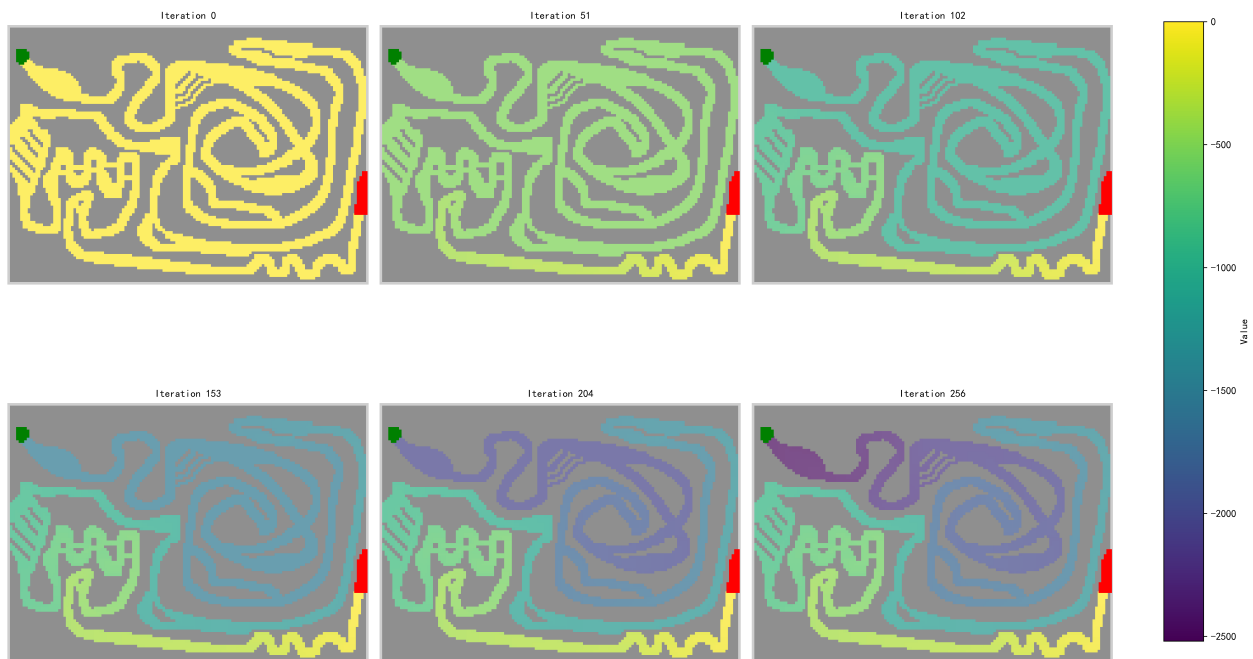


图 10: custom 赛道价值迭代价值函数演化

custom最优轨迹演化图



图 11: custom 赛道价值迭代最优轨迹演化

Q-learning 的学习过程则完全不同。图 12和图 13 显示学习曲线：在前半段代价居高不下（大量幕无法到达终点），约在幕数过半后代价骤降，随后缓慢趋平，表明算法逐渐掌握了成功路径。图 14 的价值热力图演化显示价值从起点区域开始逐步向外蔓延（正推）。图 15 的轨迹演化也印证了这一点：早期轨迹几乎无法到达终点，随着学习深入，探索距离逐渐增长，最终形成完整路径。

5.2 状态空间压缩与问题规模

图 16 展示了价值迭代收敛轮数以及 Q-learning 收敛幕数与可行格子数的关系（双对数坐标）。左图将两者放在一起对比，右上和右下分别为 Q-learning 和价值迭代的单独散点图。可以观察到：

- 价值迭代和Q-learning的迭代轮数随格子数增长均近似为对数线性。
- 相比于价值迭代，Q-learning 的收敛幕数增长更快。价值迭代哪怕在规模巨大的 custom 赛道上也仅需 256 轮即可收敛。
- 结合表 1 中的压缩率可知，BFS 将理论状态数压缩了 1 ~ 3 个数量级，这是价值迭代轮数低的可行关键。

5.3 最优代价对比

表 2 汇总了所有赛道价值迭代与 Q-learning 主实验得到的最优代价（QL 多次运行均值及标准差，因标准差为 0 故省略）。可以看出，Q-learning 在预设幕数下总能达到与价值迭代完全相同的最优代价，验证了算法的有效性。价值迭代的时间远小于 Q-learning 的训练时间（即使在 custom 上 VI 仅需 4.7 分钟，而 QL 需要约 3 小时）。

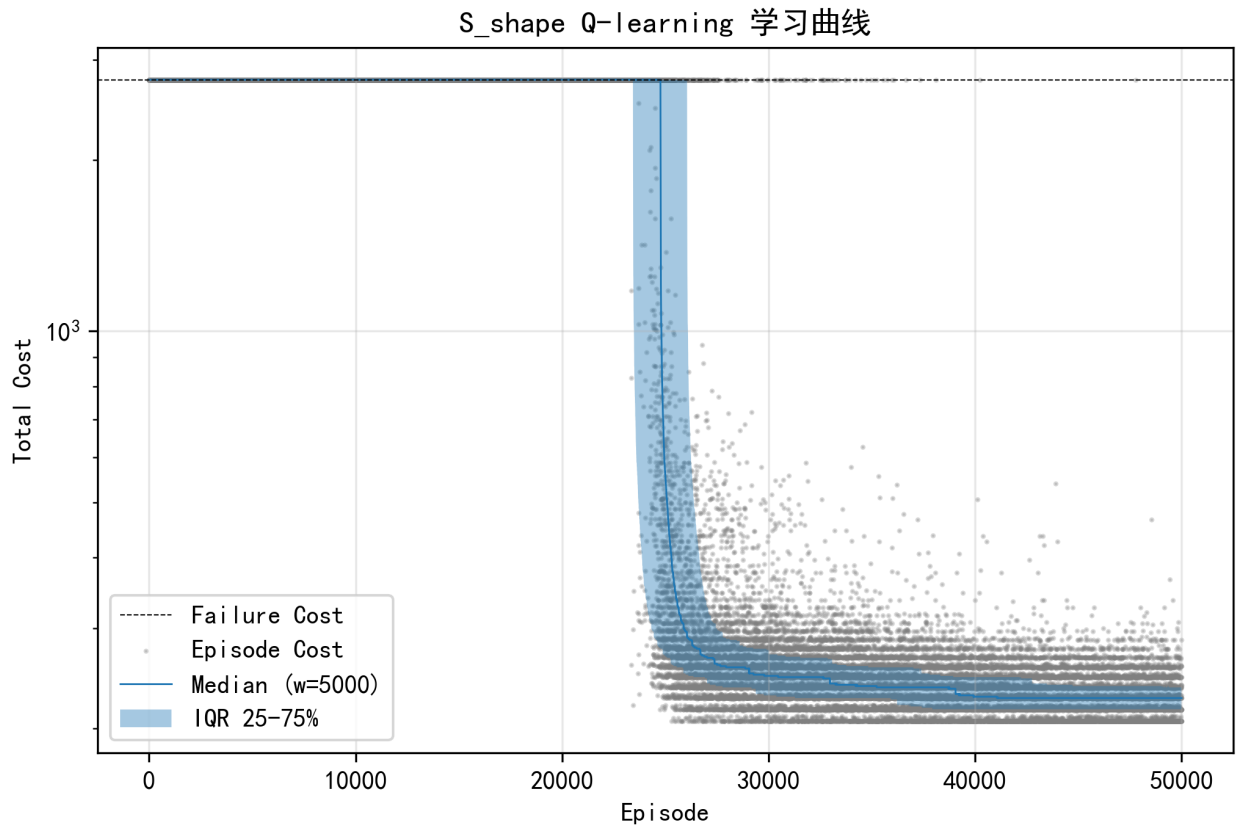


图 12: S_shape 赛道 Q-learning 学习曲线

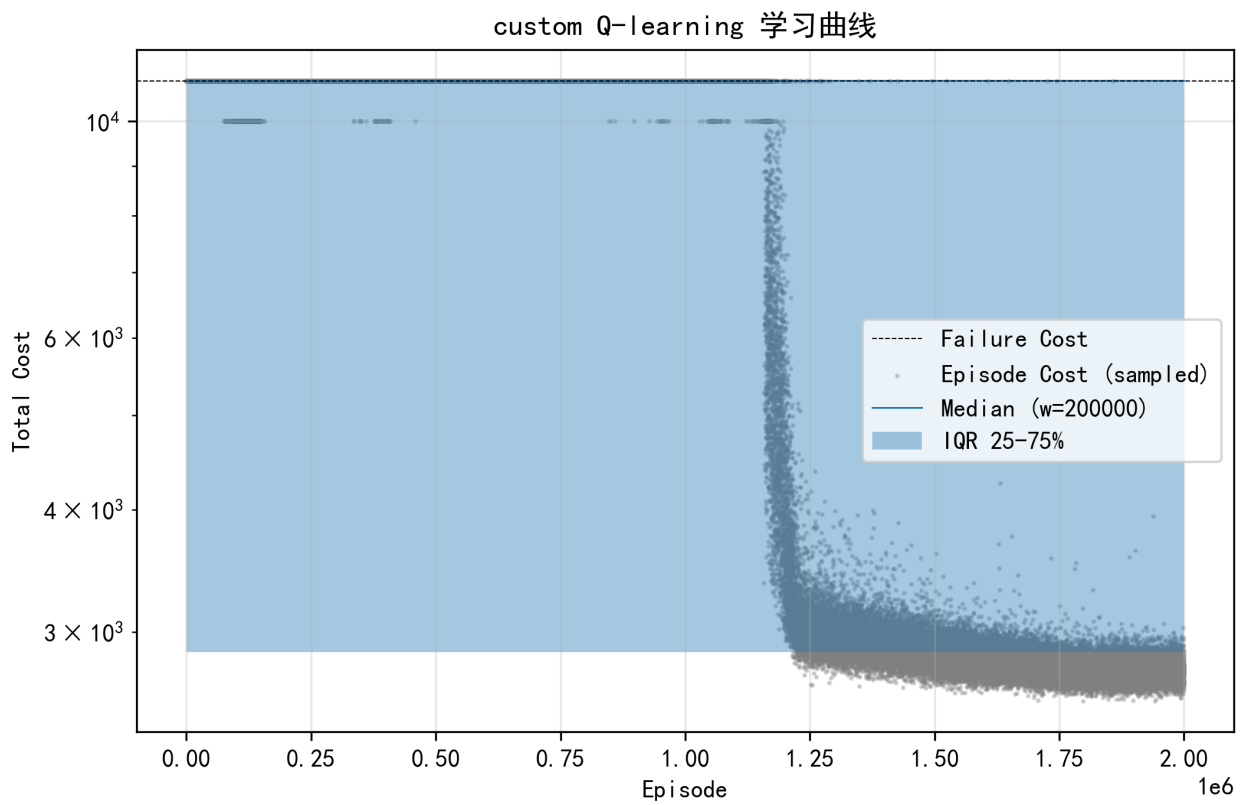


图 13: custom 赛道 Q-learning 学习曲线

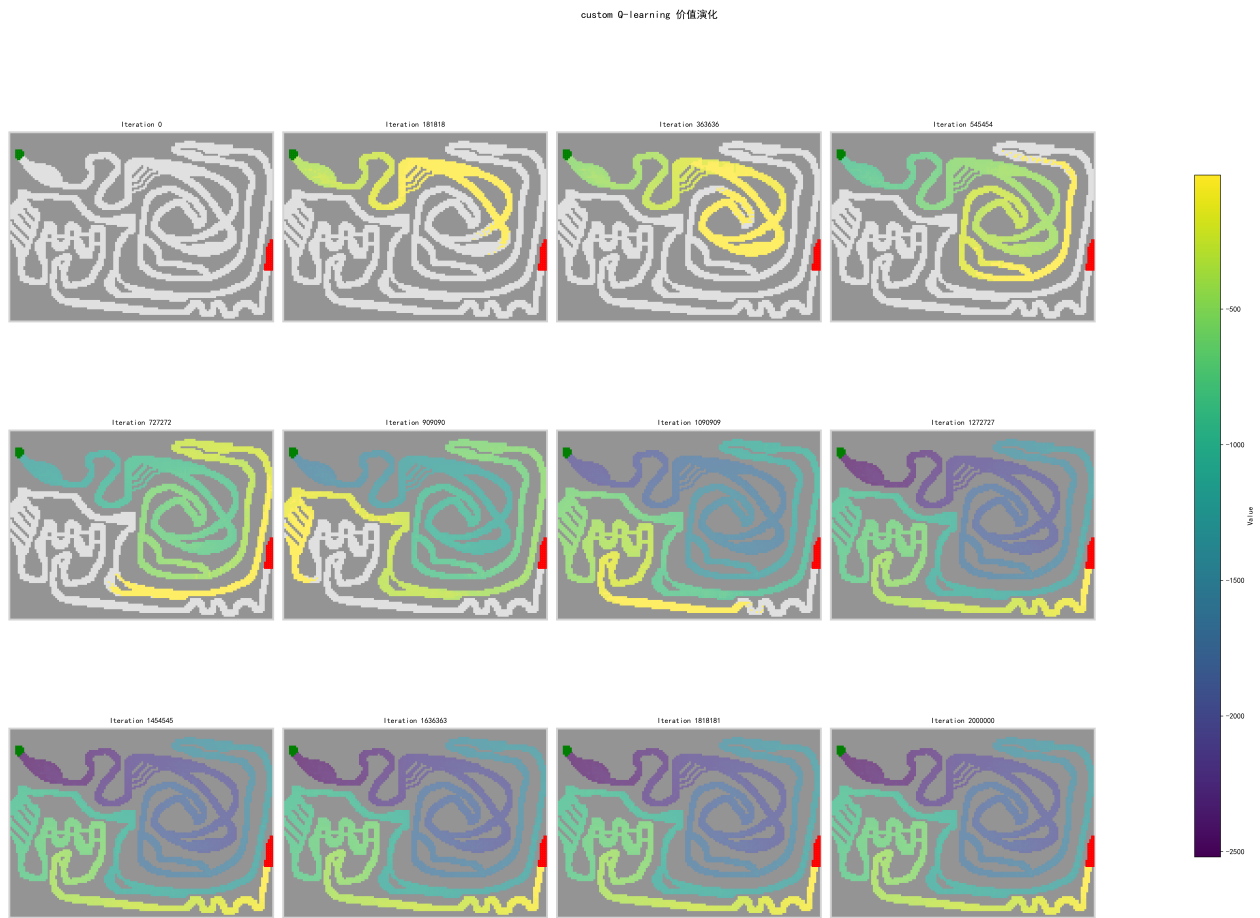


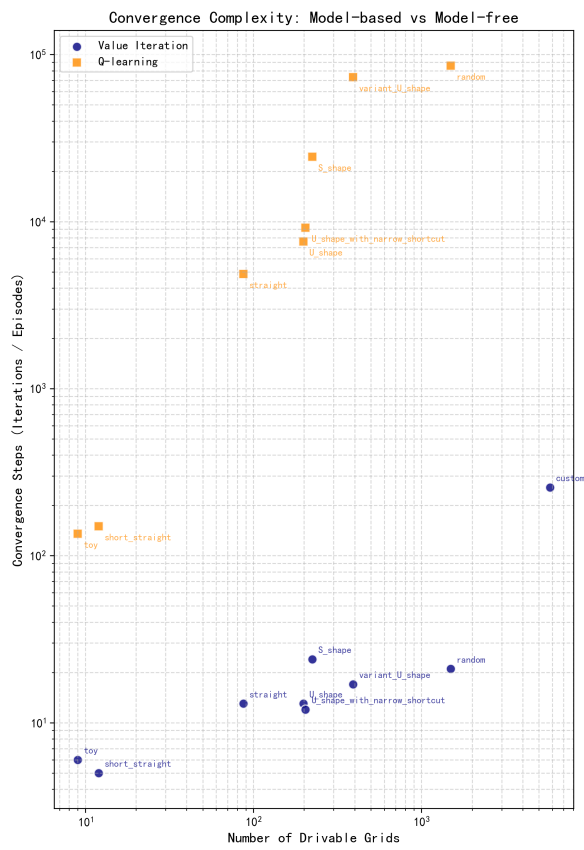
图 14: custom 赛道 Q-learning 价值热力图演化

表 2: 最优代价与训练时间对比（主实验）

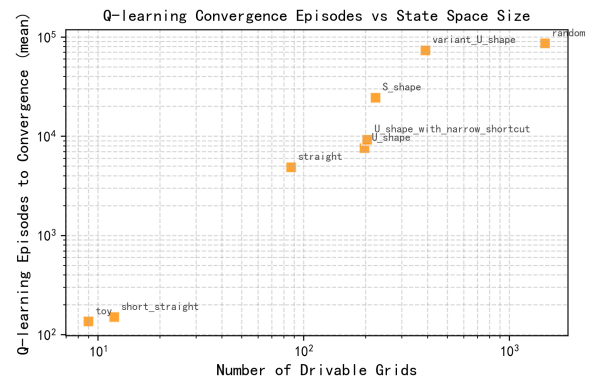
赛道	VI 代价	VI 时间(ms)	QL 预设幕数	QL 代价	QL 时间(ms)	QL 重复次数
toy	40.00	2	300	40.00	22 ± 4	10
straight	110.00	48	8000	110.00	1130 ± 17	5
short_str	30.00	1	300	30.00	17 ± 3	10
U_shape	105.00	184	30000	105.00	5729 ± 1539	5
U_shortcut	100.00	368	35000	100.00	7092 ± 1034	5
S_shape	206.25	526	50000	206.25	15999 ± 1100	5
variant_U	104.00	1615	150000	104.00	35700 ± 6053	3
random	137.50	5268	200000	137.50	70401 ± 6320	3
custom	2509.38	282660	2000000	2509.38	10728577	1



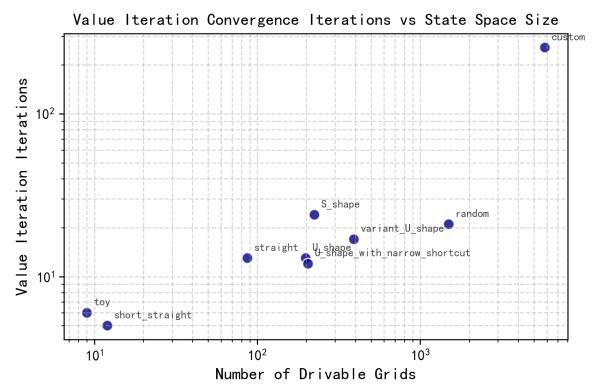
图 15: custom 赛道 Q-learning 轨迹演化



(a) VI 与 QL 收敛步数 vs 可行驶格子数 (混合)



(b) QL 收敛幕数单独



(c) VI 收敛轮数单独

图 16: 收敛步数与状态空间规模的关系

5.4 公平早停实验：样本效率与时间

为了公平比较两种算法的收敛速度，我们对前 8 个赛道运行了早停实验（custom 规模过大，未包含）。结果如表 3 所示。价值迭代轮数远小于 Q-learning 的收敛幕数（例如 U_shape 上 VI 13 轮，QL 约 7595 幕），相差约 2~3 个数量级。即使排除检测开销，QL 的纯训练时间也远大于 VI 的时间。

图 17 以对数柱状图直观对比了两者的收敛步数。图 18 和 19 分别展示了 Q-learning 收敛幕数和时间的箱线图，可见多次运行的稳定性。

表 3: 公平早停实验：收敛步数与纯训练时间

赛道	VI 轮数	VI 时间(ms)	QL 收敛幕数	QL 时间(ms)
toy	6	2	135 ± 8	7 ± 1
straight	13	48	4850 ± 40	630 ± 34
short_straight	5	1	150 ± 25	9 ± 3
U_shape	13	184	7595 ± 818	680 ± 110
U_narrow_shortcut	12	368	9196 ± 1454	919 ± 313
S_shape	24	526	24412 ± 259	5218 ± 327
variant_U_shape	17	1615	73102 ± 4961	8697 ± 904
random	21	5268	85753 ± 73	16161 ± 367
custom	256	282660	—	—

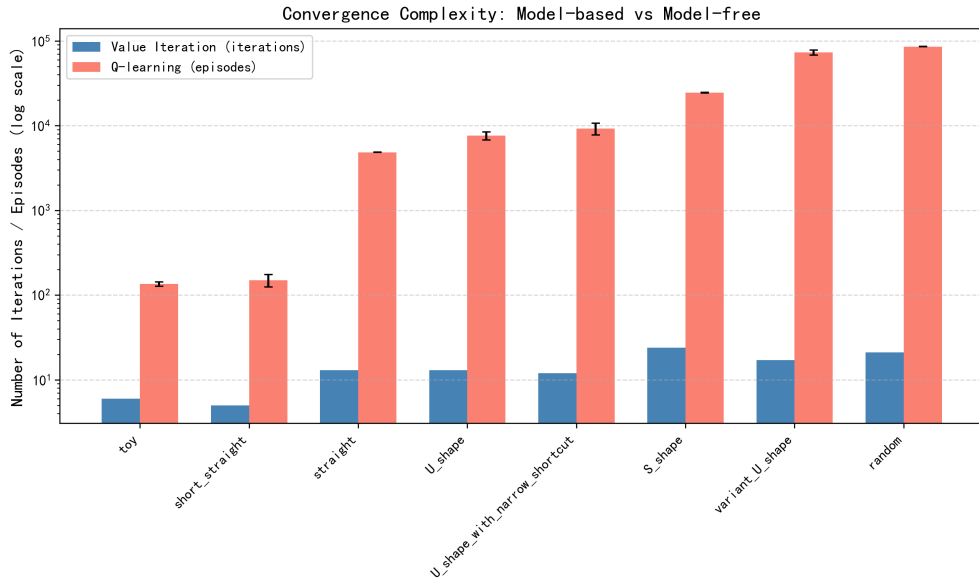


图 17: 价值迭代轮数与 Q-learning 收敛幕数对比（对数纵轴）

5.5 消融实验：死亡动作过滤

在 U_narrow_shortcut 赛道上，我们对探索策略进行了消融实验，结果见表 4。过滤（filter）模式的收敛幕数约为 8801，不过滤（no_filter）模式约为 10357，幕数减少了约 15%，证明过滤死亡动作有效避免了无效探索，提高了样本效率。然而，训练时间上两者并无显著差异（filter 平均 1074 ms，no_filter 平均 946 ms，filter 反而略高）。主要原因在于：

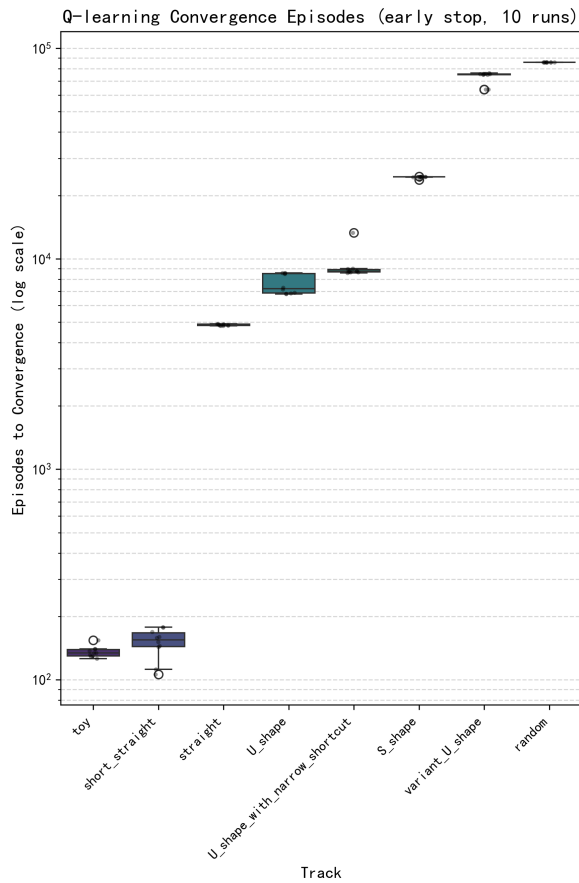


图 18: QL 收敛幕数箱线图

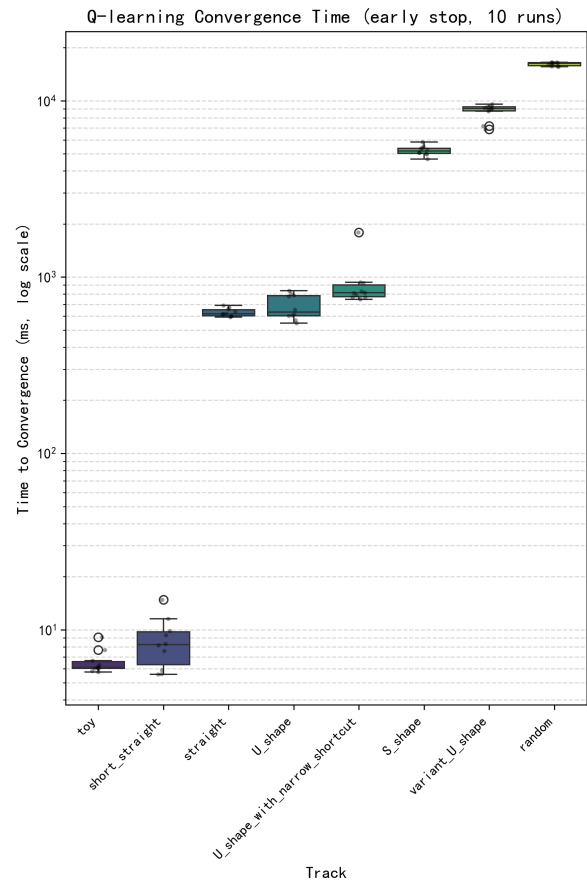


图 19: QL 收敛时间箱线图

- no_filter 模式中大量幕在第一步或前几步就撞墙死亡（短命幕），每幕平均步数极少，因此总交互步数（幕数 $\times$ 平均步长）可能并不比 filter 模式高。
- filter 模式每步需要执行 `np.where(q>-np.inf)` 来排除死亡动作，引入额外计算开销。

表 4: 消融实验结果（U_narrow_shortcut, 50 次重复）

模式	收敛幕数（均值 $\pm$ 标准差）	训练时间（ms, 均值 $\pm$ 标准差）	最终代价
filter	8801 $\pm$ 126	1074 $\pm$ 86	100.0
no_filter	10357 $\pm$ 178	946 $\pm$ 140	100.0

图 20 和 21 的箱线图展示了两种模式在幕数和时间上的分布。filter 模式的收敛所需幕数显著少于 no_filter 模式，filter 模式训练时间比 no_filter 模式略长。filter 模式的训练时间分布相较于 no_filter 模式更窄，说明经过过滤后学习过程的随机性有所降低。

综上，死亡动作过滤策略能够减少无效探索，提升样本效率，但训练时间未必节省。这揭示了样本效率与计算效率之间的权衡。

6 讨论

6.1 局限性

本研究存在以下局限性：

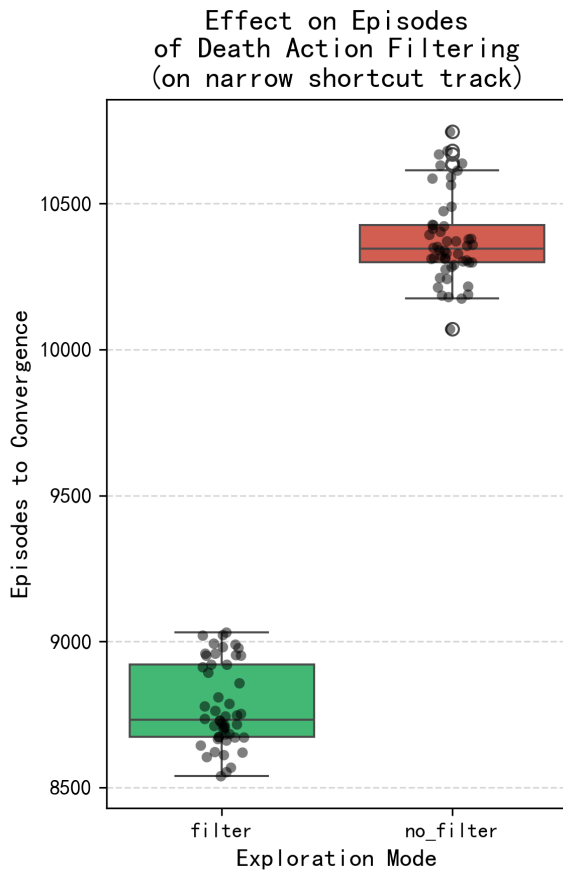


图 20: 消融实验: 收敛幕数箱线图

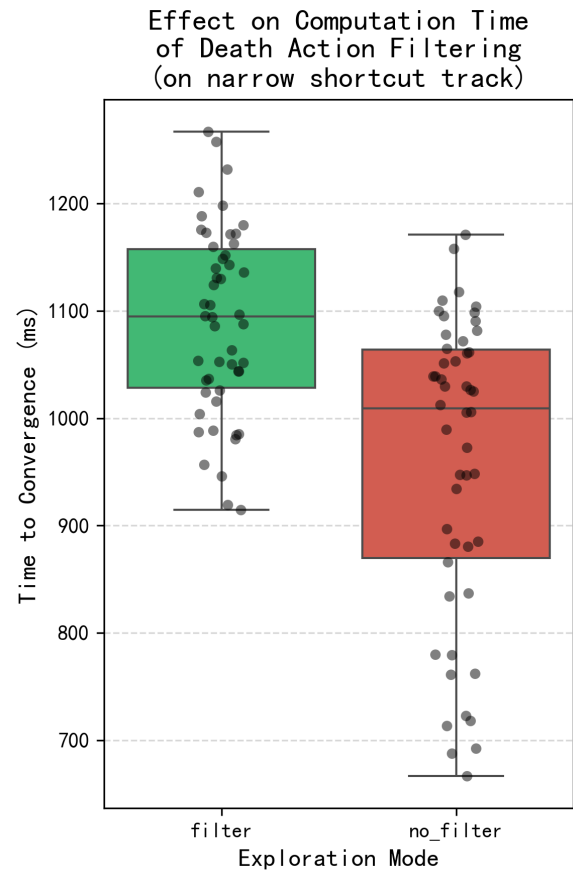


图 21: 消融实验: 收敛时间箱线图

1. **仅使用确定性环境:** 实际任务中往往存在随机性 (如风、传感器噪声)。在随机环境下, Q-learning 的 $-\infty$ 奖励需要谨慎处理 (改用极大负数), 且收敛性质可能变化。
2. **未测试其他强化学习算法:** 本文只对比了价值迭代和 Q-learning。其他算法如 SARSA (on-policy)、资格迹 (Sarsa(λ))、深度 Q 网络 (DQN) 等可能表现出不同的特性。例如:
 - SARSA 在 on-policy 更新下会更保守, 适合高风险场景 (如真实机器人操作)。
 - DQN 能够处理高维状态输入 (如图像), 适用于复杂的端到端控制任务。
3. **预设幕数的选择依赖预实验:** 主实验中 Q-learning 的幕数是根据预实验人为设定的上界, 并非自动收敛。这可能导致在某些赛道上浪费了计算资源。未来可引入自适应早停机制。
4. **消融实验仅在一个赛道上进行:** 死亡动作过滤的效果可能随赛道结构变化。虽然我们在这里选取的赛道 (即 U_shape_with_narrow_shortcut) 确实具有代表性, 但在其他类型赛道上结论可能需要进一步验证。

6.2 未来工作

基于上述局限性, 我们提出以下未来研究方向:

1. **引入随机性:** 在环境中增加随机风、滑移或观测噪声, 研究算法的鲁棒性, 并评估死亡动作过滤在随机环境下的表现。
2. **对比更多算法:** 实现 SARSA 和 Sarsa(λ), 探讨 on-policy 算法在惯性赛车中的行为 (尤其是在窄道中的保守性), 并与 Q-learning 进行系统比较。

3. **扩展至深度强化学习：**使用 DQN 处理高维状态（如直接将赛道图像作为输入），探索深度学习在惯性赛车任务上的潜力。
4. **自适应早停：**在主实验中嵌入自动早停机制（如根据最近 N 幕最优代价变化判断收敛），减少不必要的计算。
5. **多智能体与对抗环境：**考虑多辆赛车同时行驶或对抗性场景，进一步延伸问题的复杂性。

7 结论

本文针对惯性赛车问题，实现了一个具有动量特性的网格世界环境，并系统对比了有模型方法（价值迭代）与无模型方法（Q-learning）的性能。主要结论如下：

1. 通过广度优先搜索（BFS）从初始状态构建可达状态空间，将理论状态数从 H^2W^2 量级压缩至原来的 0.64% ~ 32.10%（压缩率 67.90% ~ 99.36%），使得大规模赛道（custom，可行驶格子数 5823）的价值迭代成为可能。
2. 价值迭代收敛极快（custom 赛道仅需 256 轮，约 4.7 分钟），而 Q-learning 在给定足够幕数下也能收敛到完全相同的最优代价，但需要约 200 万幕（约 3 小时），样本效率相差约 3 个数量级。
3. 提出的死亡动作过滤策略使 Q-learning 的收敛幕数减少约 15%，样本效率得到提升；然而，由于不过滤模式中大量短命幕降低了平均步长且过滤模式每步有额外计算开销，训练时间并未节省，揭示了样本效率与计算效率之间的权衡。
4. 学习动态分析表明：价值迭代的值函数从终点向起点传播（倒推），策略突变式收敛；Q-learning 的值函数从起点向终点蔓延（正推），轨迹逐步向前探索。

未来工作将引入随机环境、对比 SARSA 等 on-policy 算法、扩展至深度强化学习（DQN），并探索多智能体对抗场景。

A 附录

A.1 赛道布局图和最优路径

正文中已展示 custom、U_shape 和 S_shape 的赛道布局。为了完整起见，以下给出其余 6 个赛道（toy、straight、short_straight、U_narrow_shortcut、variant_U_shape、random）的布局图（图 22）和除了 custom 以外的其余 8 个赛道的各起点最优路径（图 23）。图中白色为普通道路，绿色为起点，红色为终点，黑色为障碍物；终点格子上标注的数字为深入奖励值；轨迹颜色越深表示该条轨迹的代价约低，纯黑色轨迹为最优路径。

A.2 消融实验原始数据（ablation_death_filter.csv）

表 5 展示了消融实验中 filter 与 no_filter 模式各 50 次重复的原始数据（收敛幕数、训练时间、最终代价）。

表 5: 消融实验完整数据（50次重复）

模式	重复编号	收敛幕数	训练时间(ms)	最终代价
filter	1	8922	1175.9	100.0
filter	2	8715	985.5	100.0
filter	3	9023	1085.9	100.0
filter	4	8707	1106.3	100.0
filter	5	8698	1139.7	100.0
filter	6	8674	1210.7	100.0
filter	7	8540	1171.4	100.0
filter	8	8568	1025.9	100.0
filter	9	8778	1053.5	100.0
filter	10	8753	1130.6	100.0
filter	11	8730	1232.0	100.0
filter	12	9022	1004.1	100.0
filter	13	8990	1151.7	100.0
filter	14	8684	1087.7	100.0
filter	15	8952	1094.5	100.0
filter	16	8954	1052.6	100.0
filter	17	8952	1035.2	100.0
filter	18	8645	1043.8	100.0
filter	19	8682	980.6	100.0
filter	20	8672	914.6	100.0
filter	21	8622	945.9	100.0
filter	22	8672	1096.9	100.0
filter	23	8612	1257.5	100.0
filter	24	8745	1024.1	100.0
filter	25	8662	984.6	100.0
filter	26	8605	1105.6	100.0
filter	27	8712	1179.9	100.0

表 5 (续)

模式	重复编号	收敛幕数	训练时间(ms)	最终代价
filter	28	8666	987.1	100.0
filter	29	8553	988.4	100.0
filter	30	8716	1095.4	100.0
filter	31	8894	1036.8	100.0
filter	32	8748	1063.5	100.0
filter	33	8735	1044.0	100.0
filter	34	8763	1171.9	100.0
filter	35	9031	1136.0	100.0
filter	36	8857	919.2	100.0
filter	37	8720	1050.5	100.0
filter	38	8994	1015.9	100.0
filter	39	8706	1159.6	100.0
filter	40	8921	1198.0	100.0
filter	41	8960	1266.7	100.0
filter	42	8620	1051.6	100.0
filter	43	8978	1124.2	100.0
filter	44	8729	1188.4	100.0
filter	45	8810	1173.1	100.0
filter	46	8674	956.6	100.0
filter	47	8912	1143.0	100.0
filter	48	8787	1129.7	100.0
filter	49	8960	1162.7	100.0
filter	50	8982	1148.4	100.0
no_filter	1	10329	1025.3	100.0
no_filter	2	10669	1029.6	100.0
no_filter	3	10631	713.5	100.0
no_filter	4	10356	688.0	100.0
no_filter	5	10176	667.0	100.0
no_filter	6	10284	692.6	100.0
no_filter	7	10299	1065.0	100.0
no_filter	8	10475	1051.3	100.0
no_filter	9	10339	1036.4	100.0
no_filter	10	10321	1095.2	100.0
no_filter	11	10071	1061.5	100.0
no_filter	12	10405	1100.0	100.0
no_filter	13	10359	1029.6	100.0
no_filter	14	10394	1026.7	100.0
no_filter	15	10307	989.6	100.0
no_filter	16	10242	1098.7	100.0

表 5 (续)

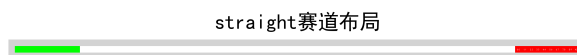
模式	重复编号	收敛幕数	训练时间(ms)	最终代价
no_filter	17	10428	972.6	100.0
no_filter	18	10681	1053.0	100.0
no_filter	19	10349	1005.3	100.0
no_filter	20	10491	1171.1	100.0
no_filter	21	10186	1071.7	100.0
no_filter	22	10371	1039.3	100.0
no_filter	23	10352	1117.5	100.0
no_filter	24	10591	1104.2	100.0
no_filter	25	10639	1109.6	100.0
no_filter	26	10276	1081.7	100.0
no_filter	27	10299	1078.0	100.0
no_filter	28	10246	1005.9	100.0
no_filter	29	10291	1012.6	100.0
no_filter	30	10311	1090.6	100.0
no_filter	31	10372	934.6	100.0
no_filter	32	10427	1060.6	100.0
no_filter	33	10378	947.1	100.0
no_filter	34	10315	947.6	100.0
no_filter	35	10341	762.0	100.0
no_filter	36	10564	885.1	100.0
no_filter	37	10330	722.7	100.0
no_filter	38	10314	865.9	100.0
no_filter	39	10311	837.1	100.0
no_filter	40	10424	896.7	100.0
no_filter	41	10302	1158.1	100.0
no_filter	42	10380	1039.2	100.0
no_filter	43	10189	883.6	100.0
no_filter	44	10587	779.8	100.0
no_filter	45	10213	761.3	100.0
no_filter	46	10414	834.1	100.0
no_filter	47	10615	948.4	100.0
no_filter	48	10217	718.2	100.0
no_filter	49	10181	780.0	100.0
no_filter	50	10746	880.6	100.0

A.3 公平早停实验原始数据 (convergence_ql.csv)

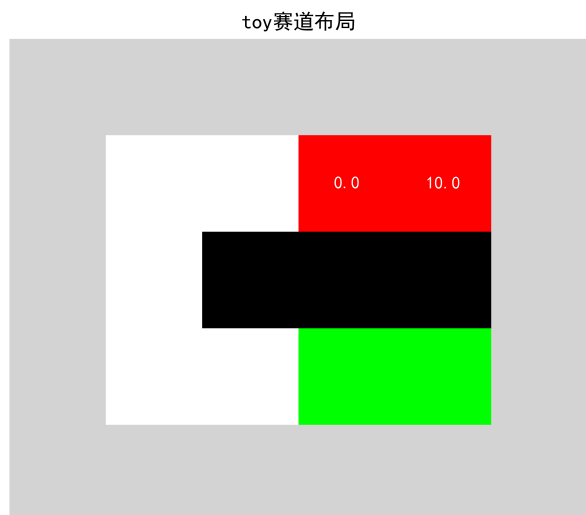
表 6 列出了每个赛道 10 次重复的收敛幕数、纯训练时间和最终代价。



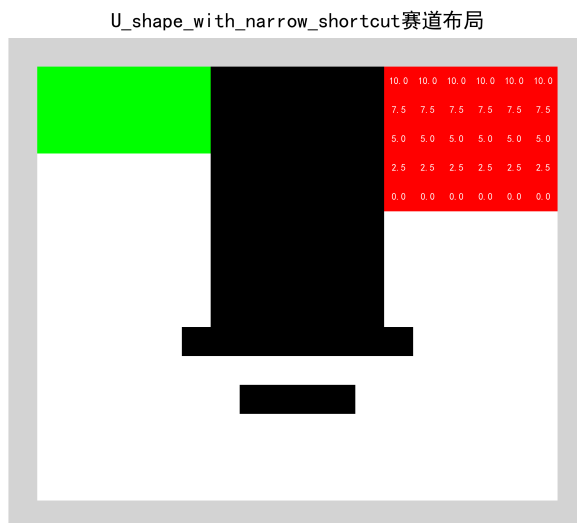
(a) short_straight 赛道布局



(b) straight 赛道布局



(c) toy 赛道布局



(d) U_narrow_shortcut 赛道布局

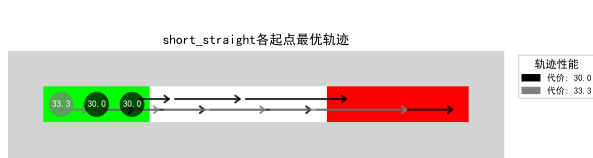


(e) variant_U_shape 赛道布局



(f) random 赛道布局

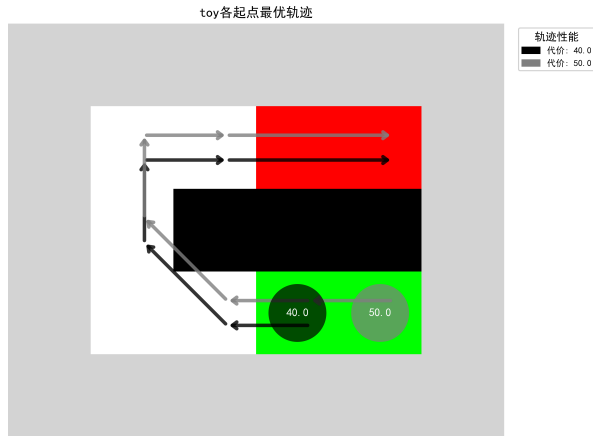
图 22: 赛道布局图



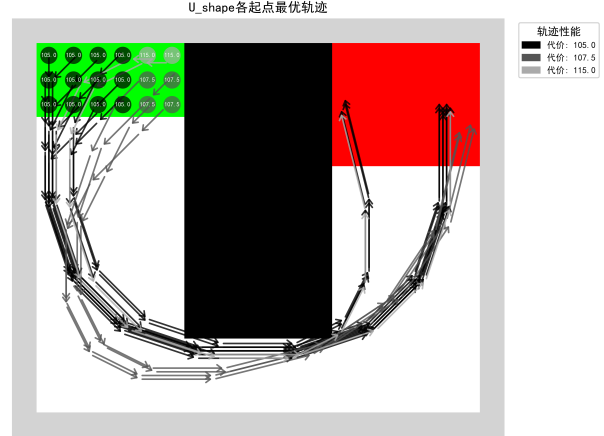
(a) short_straight 赛道所有起点最优路径



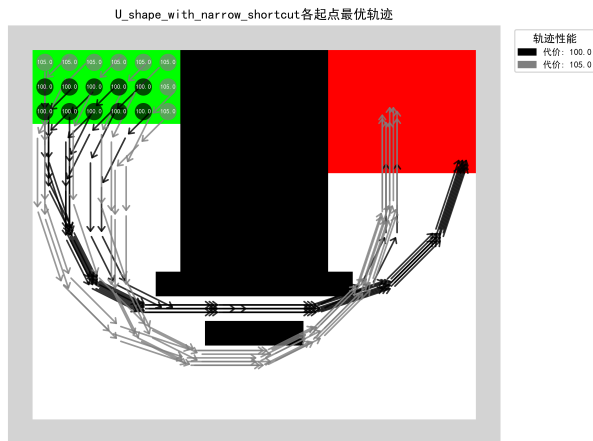
(b) straight 赛道所有起点最优路径



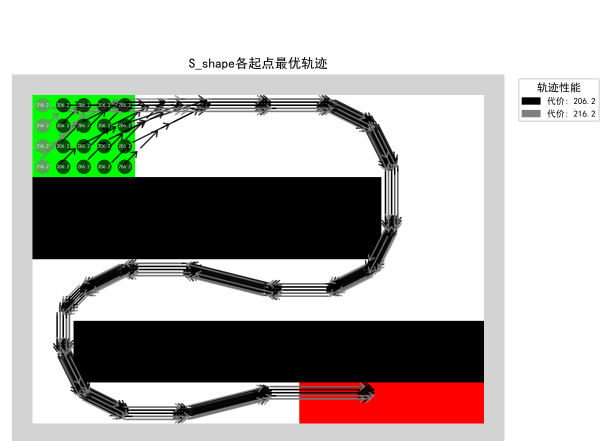
(c) toy 赛道所有起点最优路径



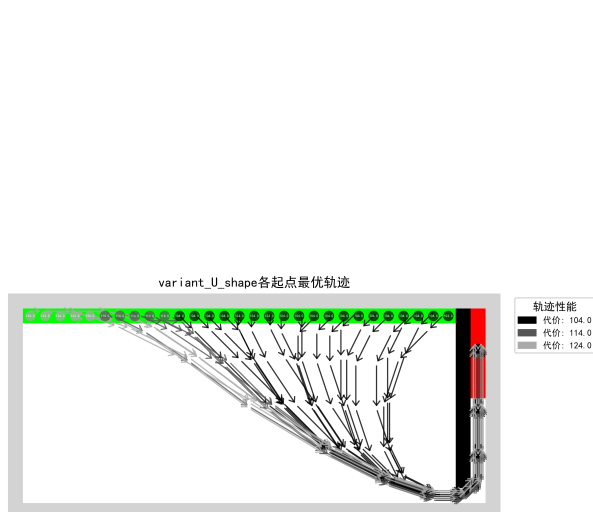
(d) U_shape 赛道所有起点最优路径



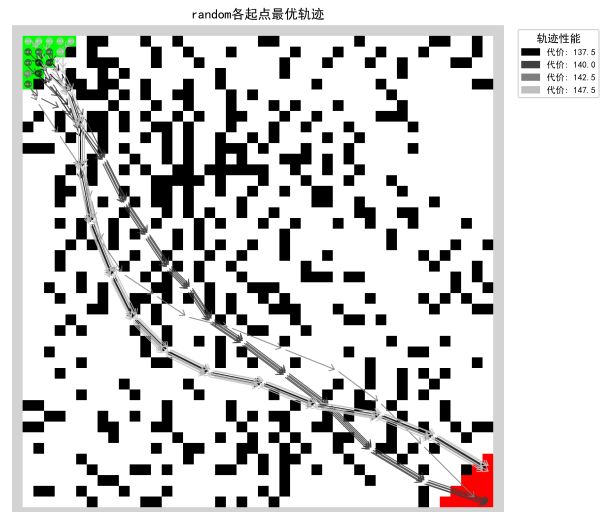
(e) U_narrow_shortcut 赛道所有起点最优路径



(f) S_shape 赛道所有起点最优路径



(g) variant_U_shape 赛道所有起点最优路径



(h) random 赛道所有起点最优路径

图 23: 赛道所有起点最优路径

表 6: 公平早停实验完整数据

赛道	重复	收敛	收敛幕数	时间(ms)	最终代价
toy	1	True	134	6.02	40.0
toy	2	True	129	5.84	40.0
toy	3	True	137	6.68	40.0
toy	4	True	134	6.06	40.0
toy	5	True	126	5.75	40.0
toy	6	True	154	7.67	40.0
toy	7	True	140	6.30	40.0
toy	8	True	140	9.08	40.0
toy	9	True	129	6.18	40.0
toy	10	True	131	6.04	40.0
straight	1	True	4821	691.73	110.0
straight	2	True	4809	600.16	110.0
straight	3	True	4825	635.16	110.0
straight	4	True	4910	599.90	110.0
straight	5	True	4796	592.39	110.0
straight	6	True	4899	619.72	110.0
straight	7	True	4863	659.15	110.0
straight	8	True	4849	612.84	110.0
straight	9	True	4833	672.51	110.0
straight	10	True	4893	619.58	110.0
short_straight	1	True	160	7.57	30.0
short_straight	2	True	169	8.30	30.0
short_straight	3	True	151	9.32	30.0
short_straight	4	True	177	14.82	30.0
short_straight	5	True	145	9.84	30.0
short_straight	6	True	106	5.58	30.0
short_straight	7	True	178	11.54	30.0
short_straight	8	True	112	5.60	30.0
short_straight	9	True	158	8.17	30.0
short_straight	10	True	143	5.92	30.0
U_shape	1	True	7280	653.42	105.0
U_shape	2	True	8561	807.77	105.0
U_shape	3	True	8464	788.80	105.0
U_shape	4	True	7167	605.91	105.0
U_shape	5	True	8524	776.42	105.0
U_shape	6	True	6821	614.42	105.0
U_shape	7	True	6807	602.36	105.0
U_shape	8	True	6856	569.35	105.0

表 6 (续)

赛道	重复	收敛	收敛幕数	时间(ms)	最终代价
U_shape	9	True	8568	838.93	105.0
U_shape	10	True	6903	547.40	105.0
U_with_narrow_shortcut	1	True	8982	813.58	100.0
U_with_narrow_shortcut	2	True	8955	815.98	100.0
U_with_narrow_shortcut	3	True	8711	767.96	100.0
U_with_narrow_shortcut	4	True	8622	747.18	100.0
U_with_narrow_shortcut	5	True	8747	768.31	100.0
U_with_narrow_shortcut	6	True	8671	791.01	100.0
U_with_narrow_shortcut	7	True	8745	830.90	100.0
U_with_narrow_shortcut	8	True	13318	1790.40	100.0
U_with_narrow_shortcut	9	True	8568	925.77	100.0
U_with_narrow_shortcut	10	True	8645	934.75	100.0
S_shape	1	True	24474	5829.65	206.25
S_shape	2	True	24501	5358.96	206.25
S_shape	3	True	24494	5064.89	206.25
S_shape	4	True	24467	5509.77	206.25
S_shape	5	True	24484	5387.98	206.25
S_shape	6	True	24587	4956.43	206.25
S_shape	7	True	24480	5078.41	206.25
S_shape	8	True	24478	5021.21	206.25
S_shape	9	True	23681	4674.93	206.25
S_shape	10	True	24475	5300.71	206.25
variant_U_shape	1	True	76388	9555.75	104.0
variant_U_shape	2	True	75683	9091.60	104.0
variant_U_shape	3	True	63771	6880.54	104.0
variant_U_shape	4	True	63686	7204.03	104.0
variant_U_shape	5	True	74802	8717.71	104.0
variant_U_shape	6	True	75838	9127.90	104.0
variant_U_shape	7	True	75401	8894.64	104.0
variant_U_shape	8	True	75102	9273.41	104.0
variant_U_shape	9	True	75372	9250.68	104.0
variant_U_shape	10	True	74981	8973.88	104.0
random	1	True	85843	16512.16	137.5
random	2	True	85707	16483.18	137.5
random	3	True	85847	15680.52	137.5
random	4	True	85687	15685.58	137.5
random	5	True	85768	16528.84	137.5
random	6	True	85752	16192.22	137.5
random	7	True	85658	16123.32	137.5

表 6 (续)

赛道	重复	收敛	收敛幕数	时间(ms)	最终代价
random	8	True	85708	15627.38	137.5
random	9	True	85859	16368.04	137.5
random	10	True	85705	16405.53	137.5

A.4 主实验原始数据 (run_all.csv)

表 7 列出了所有赛道的价值迭代与 Q-learning 主实验的完整记录 (包括多次重复)。其中 Q-learning 的收敛栏为 “N/A” (表示固定幕数训练, 非早停)。

表 7: 主实验完整数据

赛道	算法	重复	收敛	轮数/幕数	最优代价	时间(ms)
toy	Value Iteration	1	True	6	40.0	2.13
toy	Q-Learning	1	N/A	301	40.0	20.26
toy	Q-Learning	2	N/A	301	40.0	20.10
toy	Q-Learning	3	N/A	301	40.0	20.60
toy	Q-Learning	4	N/A	301	40.0	20.78
toy	Q-Learning	5	N/A	301	40.0	33.20
toy	Q-Learning	6	N/A	301	40.0	22.81
toy	Q-Learning	7	N/A	301	40.0	21.03
toy	Q-Learning	8	N/A	301	40.0	18.43
toy	Q-Learning	9	N/A	301	40.0	19.29
toy	Q-Learning	10	N/A	301	40.0	18.76
straight	Value Iteration	1	True	13	110.0	48.29
straight	Q-Learning	1	N/A	8001	110.0	1149.27
straight	Q-Learning	2	N/A	8001	110.0	1101.82
straight	Q-Learning	3	N/A	8001	110.0	1134.80
straight	Q-Learning	4	N/A	8001	110.0	1133.63
straight	Q-Learning	5	N/A	8001	110.0	1129.03
short_straight	Value Iteration	1	True	5	30.0	0.67
short_straight	Q-Learning	1	N/A	301	30.0	16.58
short_straight	Q-Learning	2	N/A	301	30.0	16.61
short_straight	Q-Learning	3	N/A	301	30.0	14.53
short_straight	Q-Learning	4	N/A	301	30.0	14.66
short_straight	Q-Learning	5	N/A	301	30.0	16.05
short_straight	Q-Learning	6	N/A	301	30.0	14.86
short_straight	Q-Learning	7	N/A	301	30.0	14.54
short_straight	Q-Learning	8	N/A	301	30.0	14.83
short_straight	Q-Learning	9	N/A	301	30.0	17.89
short_straight	Q-Learning	10	N/A	301	30.0	25.50

表 7 (续)

赛道	算法	重复	收敛	轮数/幕数	最优代价	时间(ms)
U_shape	Value Iteration	1	True	13	105.0	184.43
U_shape	Q-Learning	1	N/A	30001	105.0	4323.34
U_shape	Q-Learning	2	N/A	30001	105.0	4374.01
U_shape	Q-Learning	3	N/A	30001	105.0	5322.62
U_shape	Q-Learning	4	N/A	30001	105.0	6817.25
U_shape	Q-Learning	5	N/A	30001	105.0	7806.90
U_with_narrow_shortcut	Value Iteration	1	True	12	100.0	367.69
U_with_narrow_shortcut	Q-Learning	1	N/A	35001	100.0	6840.65
U_with_narrow_shortcut	Q-Learning	2	N/A	35001	100.0	8530.38
U_with_narrow_shortcut	Q-Learning	3	N/A	35001	100.0	5730.96
U_with_narrow_shortcut	Q-Learning	4	N/A	35001	100.0	6809.89
U_with_narrow_shortcut	Q-Learning	5	N/A	35001	100.0	7550.24
S_shape	Value Iteration	1	True	24	206.25	526.22
S_shape	Q-Learning	1	N/A	50001	206.25	16021.46
S_shape	Q-Learning	2	N/A	50001	206.25	16210.69
S_shape	Q-Learning	3	N/A	50001	206.25	14809.50
S_shape	Q-Learning	4	N/A	50001	206.25	17687.08
S_shape	Q-Learning	5	N/A	50001	206.25	15267.90
variant_U_shape	Value Iteration	1	True	17	104.0	1614.76
variant_U_shape	Q-Learning	1	N/A	150001	104.0	28773.66
variant_U_shape	Q-Learning	2	N/A	150001	104.0	38355.17
variant_U_shape	Q-Learning	3	N/A	150001	104.0	39972.32
random	Value Iteration	1	True	21	137.5	5268.46
random	Q-Learning	1	N/A	200001	137.5	75632.72
random	Q-Learning	2	N/A	200001	137.5	72191.65
random	Q-Learning	3	N/A	200001	137.5	63379.31
custom	Value Iteration	1	True	256	2509.375	282659.62
custom	Q-Learning	1	N/A	2000001	2509.375	10728576.82